Bilgisayar Mühendisliğine Giriş

Donanım ve Yazılım Temelleri

Hazırlayan:
Furkan Uzunabdullah

Tarih:
26.03.2026

Açıklama:
Bu çalışma, bilgisayar mühendisliğinin temel konularını kapsayan bireysel bir araştırma ve derleme niteliğindedir. İçerik, akademik kavramların anlaşılır ve sistematik biçimde açıklanması amacıyla hazırlanmıştır.

Beyan:
Bu doküman tarafımca hazırlanmış olup, içerikte kullanılan tüm açıklamalar, düzenlemeler ve yazım yapısı bana aittir. Başka kaynaklardan alınan bilgiler varsa uygun şekilde yorumlanmış ve özgünleştirilmiştir.

© *Furkan Uzunabdullah*

BİLGİSAYAR DONANIMI

1. Tanım:

Bilgisayar donanımı, bir bilgisayar sistemini oluşturan fiziksel ve elektronik bileşenlerin tamamıdır. Bu bileşenler; işlemci, bellek, depolama birimleri, anakart ve giriş/çıkış cihazları gibi elemanlardan oluşur ve sistemin fiziksel çalışmasını sağlar.

---

2. Temel Mantık ve Çalışma Prensibi:

Neden vardır?
Soyut yazılım komutlarının fiziksel dünyada işlenebilmesi için donanıma ihtiyaç vardır.

Hangi problemi çözer?

- *Bilginin saklanması*
- *İşlenmesi*
- *Taşınması*
- *Kullanıcı ile etkileşim*

Temel çalışma fikri:
Donanım, ***elektrik sinyallerini (0 ve 1)*** *kullanarak bilgi işler. Tüm sistem bu ikili yapı üzerine kuruludur.*

- ***Bit:*** *En küçük veri birimi (0 veya 1)*
- ***Byte:*** *8 bit*
- ***Word:*** *İşlemci mimarisine bağlı veri uzunluğu*

Elektriksel karşılık:

- *0 → düşük voltaj (~0V)*
- *1 → yüksek voltaj (~5V)*

---

3. Detaylı Çalışma Mekanizması:

Bilgisayar donanımı, katmanlı bir yapı içinde çalışır:

1. ***Fiziksel Katman (Transistörler):***
    - *Transistörler anahtar gibi davranır (açık/kapalı)*
    - *Bu durum binary veriyi temsil eder*
2. ***Mantık Kapıları:***
    - *AND (VE), OR (VEYA), NOT (DEĞİL)*
    - *Boolean cebiri ile modellenir*
3. ***Dijital Devreler:***

    - *Kombinasyonel devreler (anlık girişe bağlı)*
    - *Sıralı devreler (geçmiş duruma bağlı)*
4. ***Bileşen Seviyesi:***
    - *CPU, RAM, disk vb.*
5. ***Sistem Seviyesi:***
    - *Tüm bileşenler veri yolları ile bağlanır*

Veri akışı:
Giriş → İşleme → Depolama → Çıkış

---

4. Teknik Detaylar ve İç Yapı:

- ***Transistör:*** *Temel yapı taşı*
- ***Entegre Devre (IC):*** *Çok sayıda transistörün birleşimi*
- ***Veri yolları (Bus):***
    - *Data Bus (veri taşır)*
    - *Address Bus (adres taşır)*
    - *Control Bus (kontrol sinyalleri)*

---

5. Örnek (Somutlaştırma):

Bir kullanıcı klavyeden "A" tuşuna bastığında:

1. *Klavye sinyal üretir*
2. *Bu sinyal binary veri olarak sisteme iletilir*
3. *CPU bu veriyi işler*
4. *RAM'de geçici olarak saklanır*
5. *Ekranda görüntülenir*

---

6. Avantajlar / Sınırlamalar:

Avantajlar:

- *Yüksek hızda işlem*
- *Paralel çalışma imkânı*

Sınırlamalar:

- *Fiziksel sınırlar (ısı, enerji)*
- *Donanım maliyeti*

MERKEZİ İŞLEM BİRİMİ (MİB / CPU)

1. Tanım:

Merkezi İşlem Birimi (MİB), bilgisayar sisteminde program komutlarını doğrudan makine dili seviyesinde işleyen; aritmetik, mantıksal ve kontrol işlemlerini gerçekleştiren programlanabilir elektronik birimdir.

---

2. Temel Mantık ve Çalışma Prensibi:

Neden vardır?
Bilgisayarın temel amacı veri işlemek ve anlamlı sonuçlar üretmektir. Bu işlemleri gerçekleştirecek merkezi bir birime ihtiyaç vardır.

Hangi problemi çözer?

- *Komutların yorumlanması*
- *Verilerin işlenmesi*
- *İşlem sırasının kontrol edilmesi*

Temel çalışma fikri:
CPU, ***basit işlemleri (toplama, karşılaştırma, mantık işlemleri)*** *art arda ve belirli bir sırada çalıştırarak karmaşık işlemler oluşturur.*

---

3. Detaylı Çalışma Mekanizması (Kritik Kısım):

A. Bit → Byte → Veri Temsili

- *1 bit = 0 veya 1*
- *8 bit = 1 byte*
- *Tüm işlemler bu binary yapı üzerinden yürütülür*

B. Transistör → Mantık Kapıları → Devreler

1. ***Transistörler:***
    - *Açık/Kapalı anahtar gibi davranır*
    - *Binary sistemin fiziksel karşılığıdır*
2. ***Mantık Kapıları:***
    - *AND (VE), OR (VEYA), NOT (DEĞİL)*
    - *Boolean cebiri ile ifade edilir*
3. ***Doğruluk Tabloları:***

    - *Giriş-çıkış ilişkisini belirler*
    - *Örn: AND kapısı → sadece her iki giriş 1 ise 1 üretir*

C. Mantık Devreleri

1. Kombinasyonel Devreler:

- *Çıktı sadece o anki girdiye bağlıdır*
- *Örnek: Toplayıcı (adder)*

2. Sıralı (Ardışıl) Devreler:

- *Çıktı hem geçmiş hem mevcut girdiye bağlıdır*
- *Flip-flop içerir*
- *Saat (clock) sinyali ile çalışır*

→ Bu yapı CPU'nun "hafızaya sahip olmasını" sağlar

D. Komut İşleme Süreci (Instruction Cycle)

1. ***Fetch:*** *Komut bellekten alınır*
2. ***Decode:*** *Komut çözülür*
3. ***Execute:*** *İşlem yapılır*
4. ***Store:*** *Sonuç yazılır*

E. Komut Kümesi (Instruction Set Architecture - ISA)

- *Her işlemcinin kendine özgü komut kümesi vardır*
- *Komutlar binary (makine dili) olarak ifade edilir*

Örnek:

- *Makine dili:* `0xFB`
- *Assembly:* `ADD`

F. Assembly ve Makine Dili İlişkisi

- *Makine dili: CPU'nun doğrudan anladığı dil*
- *Assembly: İnsan tarafından okunabilir hali*
- ***Assembler:*** *Assembly → makine kodu çevirir*

G. CPU İç Yapısı

1. ***ALU (Arithmetic Logic Unit):***
    - *Aritmetik ve mantık işlemleri*
2. ***Kontrol Birimi:***
    - *Komut akışını yönetir*
3. ***Saklayıcılar (Registerlar):***
    - *Çok hızlı, küçük bellekler*
    - *Geçici veri tutar*

---

4. Teknik Detaylar ve İç Yapı:

- *Boolean cebiri → mantık işlemlerinin matematiksel temeli*
- *Clock sinyali → işlemleri senkronize eder*
- *Pipeline → paralel komut işleme*
- *Register → en hızlı veri erişim noktası*

---

5. Örnek (Adım Adım):

İşlem: `A = B + C`

1. *B ve C RAM'den register'a alınır*
2. *ALU toplama yapar*
3. *Sonuç register'a yazılır*
4. *A değişkenine aktarılır*

---

6. Avantajlar / Sınırlamalar:

Avantajlar:

- *Programlanabilir yapı*
- *Yüksek işlem hızı*
- *Karmaşık işlemleri basit adımlarla çözebilme*

Sınırlamalar:

- *Komut setine bağımlılık*
- *Isı ve enerji tüketimi*
- *Bellek erişim gecikmesi*

BELLEK

1. Tanım:

Bellek, program komutlarının, verilerin ve işlem sonuçlarının geçici veya kalıcı olarak saklandığı donanım bileşenidir.

---

2. Temel Mantık ve Çalışma Prensibi:

Neden vardır?
CPU sürekli veri üretir ve kullanır; bu verilerin saklanacağı bir ortama ihtiyaç vardır.

Hangi problemi çözer?

- *Veri saklama*
- *Programların çalıştırılması*
- *Ara sonuçların tutulması*

Temel fikir:
Hız ve kapasite arasında denge kuran ***hiyerarşik bellek yapısı***

---

3. Detaylı Çalışma Mekanizması:

A. RAM (Ana Bellek)

- *Rastgele erişim (Random Access)*
- *Adresleme ile erişilir*
- *Elektrik kesilince veri silinir (volatile)*

Çalışma:

1. *Adres belirlenir*
2. *Bellek hücresi seçilir*
3. *Veri okunur/yazılır*

B. ROM

- *Kalıcı bellek (non-volatile)*
- *Sistem başlangıç kodlarını içerir*

Türleri:

- *ROM*
- *EPROM (UV ile silinir)*
- *EEPROM (elektrikle silinir)*

C. İkincil Bellek (Diskler)

Manyetik disk çalışma süreci:

1. *Veri bloklara ayrılır*
2. *Disk plakalarına yazılır*
3. *Okuma kafası doğru iz ve sektöre gider*
4. *Veri okunur*

Erişim zamanı =

- *Kafa hareket süresi (seek time)*
- *Dönme gecikmesi*

D. Optik Disk (CD/DVD)

- *Lazer ile veri okuma*
- *Optik izler üzerinden veri saklama*

E. Bellek Hiyerarşisi

Hız → yavaş
Kapasite → büyük

1. *Register*
2. *Cache*
3. *RAM*
4. *Disk*
5. *Yedekleme birimleri*

---

4. Teknik Detaylar:

- *DRAM: Yoğun ama yavaş*
- *SRAM: Hızlı ama pahalı*
- *Paralel veri transferi*
- *Adresleme genişliği → erişilebilir bellek miktarını belirler*

---

5. Örnek (Adım Adım):

Bir program çalıştırılırken:

1. *Diskten RAM'e yüklenir*
2. *CPU komutları RAM'den çeker*
3. *Ara sonuçlar RAM'de tutulur*
4. *Sonuç tekrar diske yazılır*

---

6. Avantajlar / Sınırlamalar:

RAM:

- ○ *Çok hızlı*
- *− Uçucu*

Disk:

- ○ *Büyük kapasite*
- *− Yavaş*

GİRİŞ / ÇIKIŞ BİRİMLERİ (I/O)

1. Tanım:

Giriş/Çıkış (G/Ç) birimleri, bilgisayar sisteminin dış dünya ile veri alışverişini sağlayan; verilerin sisteme alınması, işlenmek üzere belleğe aktarılması ve sonuçların dış ortama iletilmesini gerçekleştiren donanım bileşenleridir.

---

2. Temel Mantık ve Çalışma Prensibi:

Neden vardır?
Bilgisayarlar tek başına anlamlı değildir; kullanıcıdan veri almalı ve sonuç üretmelidir.

Hangi problemi çözer?

- *Kullanıcıdan veri alma*
- *Harici cihazlarla iletişim*
- *İşlenen veriyi kullanıcıya sunma*

Temel çalışma fikri:
*G/Ç birimleri, **donanım + sürücü + işletim sistemi** katmanları üzerinden veri alışverişi yapar.*

---

3. Detaylı Çalışma Mekanizması (Kritik Kısım):

A. Giriş Süreci (Input Flow)

1. ***Fiziksel olay:***
    - *Örn: Klavye tuşuna basılması*
2. ***Sinyal üretimi:***
    - *Cihaz bu olayı elektrik sinyaline çevirir*
3. ***Kontrol birimi (Controller):***
    - *Sinyali yorumlar*
    - *Standart veri formatına dönüştürür*
4. ***Sürücü (Driver):***
    - *Donanım ile işletim sistemi arasında köprü kurar*
5. ***İşletim sistemi:***
    - *Veriyi alır*
    - *Belleğe yerleştirir*
6. ***Uygulama:***
    - *Veriyi kullanır*

B. Çıkış Süreci (Output Flow)

1. *Uygulama sonuç üretir*
2. *İşletim sistemi veriyi hazırlar*
3. *Driver aracılığıyla cihaza gönderilir*
4. *Donanım (ör. monitör) veriyi fiziksel çıktıya dönüştürür*

C. Disklerin G/Ç Olarak Rolü

- *Diskler hem giriş hem çıkış birimidir*
- *Veri:*
    - *Okunabilir (input)*
    - *Yazılabilir (output)*

D. Veri İletim Yöntemleri

1. Seri İletim:

- *Veri bit bit gönderilir*

- *Daha az kablo, daha güvenilir*
- *Örn: USB*

2. Paralel İletim:

- *Birden fazla bit aynı anda gönderilir*
- *Daha hızlı ama karmaşık*
- *Örn: eski yazıcı portları*

E. USB Mimarisi

- *Seri iletişim kullanır*
- *Veri + güç aynı hat üzerinden gider*
- *Plug-and-play desteği vardır*

F. CPU ile Etkileşim Mekanizması

1. ***Interrupt (Kesme):***
    - *Cihaz CPU'yu uyarır*
    - *CPU işlem akışını durdurur ve I/O işlemini gerçekleştirir*
2. ***DMA (Direct Memory Access):***
    - *Cihaz, CPU'yu meşgul etmeden RAM'e veri yazar*

---

4. Teknik Detaylar ve İç Yapı:

- ***Controller (Denetleyici):*** *Cihazın beynidir*
- ***Driver:*** *Yazılım seviyesinde kontrol sağlar*
- ***Buffer:*** *Geçici veri saklama alanı*

Portlar:

- *USB*
- *Seri port (RS-232)*
- *Paralel port*

---

5. Örnek (Adım Adım):

Klavye ile "A" tuşuna basılması:

1. *Tuş sinyali oluşur*
2. *Klavye denetleyicisi bunu scan code'a çevirir*
3. *Driver bunu ASCII karaktere dönüştürür*

4. *İşletim sistemi RAM'e yazar*
5. *Uygulama bu veriyi kullanır*

---

6. Avantajlar / Sınırlamalar:

Avantajlar:

- *Sistem ile dış dünya arasında iletişim sağlar*
- *Modüler yapı (farklı cihazlar bağlanabilir)*

Sınırlamalar:

- *I/O işlemleri CPU'ya göre yavaştır*
- *Sürücü uyumsuzlukları sorun yaratabilir*

BİLGİSAYAR AĞLARI

1. Tanım:

Bilgisayar ağları, bilgisayarların ve diğer cihazların belirli protokoller aracılığıyla birbirleriyle iletişim kurmasını sağlayan bağlantı sistemleridir.

---

2. Temel Mantık ve Çalışma Prensibi:

Neden vardır?
Tek bir bilgisayarın kaynakları sınırlıdır; paylaşım ve iletişim gereklidir.

Hangi problemi çözer?

- *Veri paylaşımı*
- *Kaynak paylaşımı (yazıcı, disk)*
- *Uzaktan erişim*

Temel fikir:
Veri, belirli kurallara (protokoller) göre paketlenir ve iletilir.

---

3. Detaylı Çalışma Mekanizması (Kritik Kısım):

A. Veri İletim Süreci (TCP/IP Modeli)

1. ***Uygulama katmanı:*** *Veri oluşturulur*
2. ***Taşıma katmanı (TCP):***
    - *Veri parçalara ayrılır*
    - *Hata kontrolü yapılır*
3. ***Ağ katmanı (IP):***
    - *Paketlere adres eklenir*
4. ***Veri bağlantı katmanı:***
    - *Frame oluşturulur*
5. ***Fiziksel katman:***
    - *Bitler elektrik sinyali olarak iletilir*

B. Protokoller

- ***TCP/IP:*** *İnternetin temel protokolü*
- ***NETBIOS, AppleTalk, NetWare:*** *Alternatif protokoller*

C. İstemci / Sunucu Mimarisi

Sunucu:

- *Güçlü sistem*
- *İşlemleri gerçekleştirir*

İstemci:

- *Kullanıcı arayüzünü sağlar*

Çalışma:

1. *İstemci istek gönderir*
2. *Sunucu işler*
3. *Sonuç geri gönderilir*

D. Ağ Programlama

- *Farklı sistemlerde çalışan programlar iletişim kurar*
- *Socket yapıları kullanılır*

E. Ağ Türleri

- ***LAN (Local Area Network):*** *Küçük alan*
- ***WAN (Wide Area Network):*** *Geniş alan*

F. Çok Kullanıcılı Sistemler

- *Ana bilgisayar (server)*
- *Terminaller (client)*
- *Kaynak paylaşımı yapılır*

---

4. Teknik Detaylar:

- *IP adresleme*
- *Paket yönlendirme (routing)*
- *Ethernet teknolojisi*
- *Yönlendirici (router)*

---

5. Örnek (Adım Adım):

Bir web sayfası açılırken:

1. *Tarayıcı HTTP isteği oluşturur*
2. *TCP ile paketlenir*
3. *IP ile adreslenir*
4. *Router üzerinden iletilir*
5. *Sunucu yanıt verir*
6. *Veri ekranda gösterilir*

---

6. Avantajlar / Sınırlamalar:

Avantajlar:

- *Kaynak paylaşımı*
- *Ölçeklenebilirlik*
- *Dağıtık sistemler*

Sınırlamalar:

- *Güvenlik riskleri*
- *Gecikme (latency)*
- *Protokol bağımlılığı*

BİLGİSAYAR YAZILIMI

1. Tanım:

Bilgisayar yazılımı, bilgisayar veya sayısal sistemler üzerinde belirli bir görevi gerçekleştirmek amacıyla oluşturulmuş; programlar, veriler ve bu programların çalışmasını sağlayan komutlar bütünüdür. Yazılım, donanım dışında kalan ve sistemin işlevselliğini sağlayan temel bileşendir.

---

2. Temel Mantık ve Çalışma Prensibi:

Bilgisayar yazılımları genel olarak iki ana gruba ayrılır:

- ***Sistem yazılımları:*** *Donanımı yöneten ve diğer yazılımların çalışmasını sağlayan yazılımlar (ör. işletim sistemleri)*
- ***Uygulama yazılımları:*** *Kullanıcı ihtiyaçlarına yönelik geliştirilen yazılımlar*

Yazılımın temelinde şu dönüşüm yer alır:

Problem → Algoritma → Program → Makine Kodu → Çalıştırma

Bu süreçte:

- *Problemler algoritmalarla ifade edilir*
- *Algoritmalar programlama dilleri ile kodlanır*
- *Kodlar makine diline çevrilerek CPU tarafından yürütülür*

---

3. Detaylı Çalışma Mekanizması (Kritik Kısım):

A. Program ve Program Kodu

- ***Program:*** *Belirli bir görevi yerine getiren algoritmik yapının uygulanmış halidir*
- ***Program kodu:*** *Bu programın bir programlama dili ile yazılmış halidir*

Programlar tek başına olabileceği gibi, büyük sistemlerde birden fazla programın birleşmesiyle yazılım sistemini oluşturur.

B. Değişkenler (Variables)

Tanım:
Bellekte veri saklamak için kullanılan isimlendirilmiş alanlardır.

Çalışma Mekanizması:

- *Her değişken bellekte bir adresi temsil eder*
- *Değer bu adrese yazılır*
- *Değişken, aslında bir **bellek etiketi** olarak düşünülebilir*

Örnek:
Bir programda 5 farklı veri tutulacaksa 5 farklı değişken gerekir.

C. Operatörler

Tanım:
Değişkenler veya veriler üzerinde işlem yapılmasını sağlayan sembollerdir.

Türleri:

- *Aritmetik:* `+ - * /`
- *Mantıksal:* `&& || !`
- *Karşılaştırma:* `== < >`

Çalışma Mantığı:
*Operatörler CPU'nun **ALU (Aritmetik Mantık Birimi)** tarafından gerçekleştirilir.*

D. Karakter Temsili (ASCII / Unicode)

Bilgisayarlar metinleri doğrudan değil, sayısal karşılıklarıyla saklar.

Örnek:

- *A → 65*
- *B → 66*

"AB" → bellekte `65 66` *olarak tutulur*

*Bu dönüşüm **karakter tabloları** aracılığıyla yapılır.*

E. String (Sözce)

- *Karakter dizileridir*
- *Bellekte ardışık byte'lar halinde tutulur*

Önemli ayrım:

- `1972` *→ sayısal veri*

- `"1972"` → *string (karakter dizisi)*

String veriler doğrudan aritmetik işlemlere sokulamaz.

F. İfadeler (Expressions / Deyimler)

- *Değişkenler ve operatörlerin birleşimiyle oluşur*
- *Tek bir işlem adımını temsil eder*

Örnek:
`x + y`, `a > b`

G. Atama İşlemi

- *Bir değerin bir değişkene aktarılmasıdır*
- *Bellek üzerinde veri yazma işlemi gerçekleşir*

Örnek:

- `N ← 5`
- `Toplam ← 15 + x`

Arka planda:

1. *CPU ifadeyi hesaplar*
2. *Sonuç register'da tutulur*
3. *RAM'de ilgili adrese yazılır*

---

4. Teknik Detaylar ve İç Yapı:

- *Bellek adresleme (memory addressing)*
- *Veri tipleri:* `int`, `char`, `string`
- *Binary veri temsili (0 ve 1)*
- *CPU–ALU işlem mekanizması*
- *Program → makine kodu dönüşümü*

---

5. Örnek (Adım Adım):

Kod:

```
int x = 5;
int y = 3;
```

int z = x + y;

Çalışma süreci:

1. *x ve y için RAM'de alan ayrılır*
2. *5 ve 3 değerleri belleğe yazılır*
3. *CPU bu değerleri register'a alır*
4. *ALU toplama işlemini gerçekleştirir*
5. *Sonuç (8) z değişkenine atanır*

---

6. Avantajlar / Sınırlamalar:

Avantajlar:

- *Modüler ve yeniden kullanılabilir yapı*
- *Karmaşık problemlerin çözümünü sağlar*
- *Donanımı esnek şekilde kullanmayı mümkün kılar*

Sınırlamalar:

- *Bellek yönetimi hataları (memory leaks vb.)*
- *Tip uyumsuzlukları*
- *Donanıma bağımlılık*

İŞLETİM SİSTEMİ

1. Tanım:

İşletim sistemi, bilgisayar donanımı ile kullanıcı ve uygulama programları arasında etkileşimi sağlayan; donanım ve yazılım kaynaklarını yöneten, paylaştıran ve sistemin genel çalışma düzenini belirleyen temel sistem yazılımıdır.

---

2. Temel Mantık ve Çalışma Prensibi:

Neden vardır?
Donanım kaynakları (CPU, bellek, disk vb.) sınırlıdır ve birden fazla program tarafından paylaşılması gerekir.

Hangi problemi çözer?

- *Kaynak yönetimi*
- *Programların koordinasyonu*
- *Kullanıcı ile donanım arasındaki soyutlama*

Temel çalışma fikri:
*İşletim sistemi, donanımı doğrudan kullanmak yerine, **soyut bir arayüz sunarak** hem kullanıcıların hem de programların donanımı güvenli ve verimli şekilde kullanmasını sağlar.*

---

3. Detaylı Çalışma Mekanizması (Kritik Kısım):

A. Çoklu Görev (Multitasking) ve CPU Yönetimi

- *İşletim sistemi CPU'yu farklı programlar arasında paylaştırır*
- *Her programa belirli bir zaman dilimi (time slice) verilir*

Süreç:

1. *Birden fazla program hazır bekler*
2. *Scheduler (zamanlayıcı) bir program seçer*
3. *CPU o programa atanır*
4. *Süre dolunca başka programa geçilir*

→ Bu işlem çok hızlı gerçekleştiği için paralel çalışıyormuş gibi görünür

---

B. Bellek Yönetimi (Memory Management)

- *Programlar doğrudan RAM'e sığmayabilir*
- *Bu durumda **sayfalama (paging)** kullanılır*

Çalışma:

1. *Program parçalara ayrılır (page)*
2. *Gerekli sayfa RAM'e yüklenir*
3. *İhtiyaç duyuldukça diğer sayfalar diskten getirilir*

Page Replacement:

- *RAM dolduğunda hangi sayfanın çıkarılacağına OS karar verir*

C. Dosya Sistemi (File System)

- *Veriler dosyalar halinde saklanır*
- *Dosyalar klasörler içinde organize edilir*

Dosya yapısı:

- *İsim (name)*
- *Uzantı (extension)*
- *Tür (type)*

Örnek:

- `oyun.c`
- `resim.jpeg`

Çalışma mantığı:

- *Disk üzerinde veri bloklar halinde tutulur*
- *Dosya sistemi bu blokların düzenini yönetir*

D. Giriş/Çıkış Yönetimi

- *Donanım cihazları doğrudan kullanılmaz*
- ***Sürücüler (drivers)** aracılığıyla kontrol edilir*

Süreç:

1. *Program I/O isteği gönderir*
2. *OS bunu sürücüye iletir*
3. *Donanım işlemi gerçekleştirir*

E. Kaynak Yönetimi

İşletim sistemi şu kaynakları yönetir:

- *CPU*
- *Bellek*
- *Disk*
- *Giriş/çıkış cihazları*

---

4. Teknik Detaylar ve İç Yapı:

- ***Process (süreç):** Çalışan program*
- ***Thread:** Sürecin alt birimi*
- ***Scheduler:** CPU zamanlaması yapar*
- ***Virtual Memory:** RAM + disk birleşimi*
- ***Interrupt:** Donanımın CPU'yu uyarması*

---

5. Örnek (Adım Adım):

Bir kullanıcı hem müzik dinlerken hem tarayıcı kullanıyorsa:

1. *OS iki programı süreç olarak başlatır*
2. *CPU zaman dilimlerine bölünür*
3. *Müzik programı kısa süre çalışır*
4. *Tarayıcı çalışır*
5. *Bu döngü sürekli tekrar eder*

→ Kullanıcıya aynı anda çalışıyormuş gibi görünür

---

6. Avantajlar / Sınırlamalar:

Avantajlar:

- *Kaynakların verimli kullanımı*
- *Çoklu görev desteği*
- *Donanım soyutlaması*

Sınırlamalar:

- *Ek yük (overhead) oluşturur*
- *Hatalı yönetim performansı düşürür*

SAYI SİSTEMLERİ

1. Tanım:

Sayı sistemleri, sayısal değerlerin belirli bir taban kullanılarak temsil edilmesini sağlayan aritmetiksel gösterim yöntemleridir.

---

2. Temel Mantık ve Çalışma Prensibi:

Neden vardır?
Farklı uygulamalarda sayıları daha verimli temsil etmek gerekir.

Hangi problemi çözer?

- *Veri temsili*

- *Sayısal işlemlerin kolaylaştırılması*

Temel fikir:
Bir sayının değeri, kullanılan tabana ve basamaklara bağlıdır.

---

3. Detaylı Çalışma Mekanizması:

A. Taban (Base) Kavramı

Bir sayı sisteminde:

- *Kullanılabilecek rakam sayısı = taban*

Örnekler:

- ***10 tabanlı (Decimal):*** *0–9*
- ***2 tabanlı (Binary):*** *0–1*
- ***16 tabanlı (Hexadecimal):*** *0–9, A–F*

B. Binary Sistem ve Bilgisayarlar

Bilgisayarlar yalnızca iki durumu ayırt edebilir:

- *0 (kapalı)*
- *1 (açık)*

Bu nedenle tüm veriler binary olarak saklanır.

C. Sayı Dönüşümleri

- *Decimal ↔ Binary*
- *Binary ↔ Hexadecimal*

Bu dönüşümler matematiksel kurallara göre yapılır.

D. Kodlama (Encoding)

- *Sayılar belirli formatlarda temsil edilir*
- *Örnek: ASCII, Unicode*

Kodlama, verinin belirli bir düzende saklanmasını sağlar.

---

4. Teknik Detaylar:

- *Bit, Byte, Word kavramları*
- *İkili aritmetik*
- *Taşma (overflow)*
- *İşaretli / işaretsiz sayılar*

---

5. Örnek (Adım Adım):

Decimal 10 sayısı:

1. *10 → 2'ye bölünür*
2. *Kalanlar yazılır*
3. *Sonuç: `1010` (binary)*

---

6. Avantajlar / Sınırlamalar:

Avantajlar:

- *Binary sistem donanımla uyumludur*
- *Hexadecimal gösterim daha okunabilir*

Sınırlamalar:

- *Binary insan için zor okunur*
- *Dönüşüm gerektirir*

---

7. Bilgisayar Mühendisliğindeki Yeri:

Sayı sistemleri, bilgisayar mimarisi, veri temsili, kriptografi ve düşük seviyeli programlamanın temelini oluşturur. Tüm dijital sistemler bu yapı üzerine kuruludur.

ALGORİTMA

1. Tanım:

Algoritma, bir problemin çözümüne yönelik olarak analiz sonucu elde edilen; açık, sonlu, sıralı ve bilgisayar programı ile ifade edilip çalıştırılabilir adımlar dizisidir.

---

2. Temel Mantık ve Çalışma Prensibi:

Bir problemin bilgisayar yardımıyla çözülebilmesi için öncelikle analiz edilmesi gerekir. Bu analiz sonucunda elde edilen çözüm yolu algoritma ile ifade edilir.

Temel amaç:

- *Problemi adım adım çözmek*
- *Bu çözümü bilgisayar tarafından uygulanabilir hale getirmek*

Algoritmaların temelinde şu yaklaşım vardır:
→ Karmaşık problemleri ***basit, sıralı ve açık adımlara bölmek***

---

3. Detaylı Çalışma Mekanizması (Kritik Kısım):

A. Algoritmanın İfade Edilmesi

Algoritmalar şu yapılarla ifade edilir:

- ***İlkeller (Primitives):***
  Temel işlem adımlarıdır (atama, karşılaştırma vb.)
- ***Sözde Kod (Pseudocode):***
  Programlama diline yakın, ancak bağımsız bir ifade biçimi
- ***Akış Diyagramları:***
  Grafiksel gösterim

Örnek:
```
if (a > b) then c ← 9
```

B. Böl ve Yönet (Divide and Conquer)

- *Problem küçük alt problemlere bölünür*
- *Alt problemler çözülür*
- *Sonuçlar birleştirilir*

Adım adım:

1. *Problem ikiye bölünür*
2. *Alt parçalar tekrar bölünür*

3. *En küçük parçalar çözülür*
4. *Sonuçlar birleştirilir*

C. Çevrimli (Iteratif) Yaklaşım

- *Belirli işlemler döngülerle tekrar edilir*
- *Programlama dillerinde: `for`, `while`*

D. Özyinelemeli (Rekürsif) Yaklaşım

- *Fonksiyon kendi kendini çağırır*
- *Alt problem çözümü üzerinden ilerler*

Not:
Rekürsif yöntem her zaman daha hızlı değildir, ancak bazı problemlerde daha doğal çözümler sağlar.

E. Algoritma Verimliliği

Zaman Karmaşıklığı (Time Complexity):

- *Algoritmanın çalışma süresi*

Alan Karmaşıklığı (Space Complexity):

- *Kullanılan bellek miktarı*

Asimptotik gösterim:

- *Big-O notasyonu kullanılır*

F. Kıyaslama (Benchmarking)

- *Aynı işi yapan algoritmalar karşılaştırılır*
- *Standart veri setleri kullanılır*
- *Performans ölçülür*

---

4. Teknik Detaylar ve İç Yapı:

- *Big-O notasyonu*
- *Döngü yapıları*

- *Rekürsiyon mekanizması*
- *Veri erişim maliyeti*
- *İşlem sayısı analizi*

---

5. Örnek (Adım Adım):

Problem: Bir diziyi sıralama

1. *Dizi ikiye bölünür*
2. *Alt diziler tekrar bölünür*
3. *Tek eleman kalınca sıralı kabul edilir*
4. *Parçalar birleştirilir*

*→ Bu süreç **merge sort** mantığıdır*

---

6. Avantajlar / Sınırlamalar:

Avantajlar:

- *Problemleri sistematik çözer*
- *Yazılım geliştirme temelidir*

Sınırlamalar:

- *Yanlış tasarım verimsizliğe yol açar*
- *Karmaşık problemlerde tasarım zorlaşır*

VERİ YAPILARI

1. Tanım:

Veri yapıları, verilerin bilgisayar belleğinde nasıl organize edildiğini, saklandığını ve erişildiğini belirleyen yapılardır.

---

2. Temel Mantık ve Çalışma Prensibi:

Veri tek başına anlamlı değildir; anlam kazanması için düzenlenmesi gerekir.

Temel amaç:

- *Veriyi organize etmek*
- *Veriye hızlı ve verimli erişim sağlamak*

Aynı veri farklı veri yapılarında farklı anlamlar taşıyabilir.

---

3. Detaylı Çalışma Mekanizması (Kritik Kısım):

A. Bellekte Veri Organizasyonu

- *Veriler bellek adreslerinde tutulur*
- *Veri yapısı bu adreslerin nasıl kullanılacağını belirler*

B. Veri Modelleri

- *Problemi temsil etmek için kullanılır*
- *Soyut bir yaklaşım sunar*

Önemli nokta:
→ Doğru veri modeli seçilmezse problem çözülemez veya verimsiz olur

---

C. Temel Veri Yapıları

1. ***Diziler (Array):***
    - *Aynı tür veriler*
    - *Bellekte ardışık saklanır*
2. ***Listeler (Linked List):***
    - *Dinamik yapı*
    - *Elemanlar birbirine bağlıdır*
3. ***Ağaçlar (Tree):***
    - *Hiyerarşik yapı*

D. Kullanıcı Tanımlı Veri Yapıları

- *Programcı yeni veri yapıları oluşturabilir*
- *Örn: struct, class*

---

4. Teknik Detaylar ve İç Yapı:

- *Bellek adresleme*
- *Statik vs dinamik yapı*
- *Veri erişim süresi*
- *Pointer kullanımı*

---

5. Örnek (Adım Adım):

Dizi kullanımı:

1. *5 elemanlı dizi tanımlanır*
2. *Bellekte ardışık alan ayrılır*
3. *Elemanlar bu alanlara yazılır*
4. *İndeks ile erişilir*

---

6. Avantajlar / Sınırlamalar:

Avantajlar:

- *Verimli veri yönetimi*
- *Algoritma performansını artırır*

Sınırlamalar:

- *Yanlış seçim performansı düşürür*
- *Bazı yapılar karmaşıktır*

PROGRAMLAMA DİLLERİ

1. Tanım:

Programlama dilleri, bilgisayarın iç yapısına doğrudan hâkim olmayı gerektirmeden, algoritmaların ifade edilmesini ve programların geliştirilmesini sağlayan, kendine özgü sözdizimi (syntax) ve kuralları (grammar) olan formal dillerdir.

---

2. Temel Mantık ve Çalışma Prensibi:

*Programlama dilleri, insanlar ile bilgisayar arasında bir **ara katman** görevi görür.*

Temel amaç:

- *Algoritmaları ifade etmek*
- *Program geliştirmeyi kolaylaştırmak*
- *Donanım karmaşıklığını gizlemek*

Her programlama dili:

- *Giriş/çıkış işlemleri*
- *Hesaplama işlemleri*
- *Kontrol yapıları (karar ve döngü)*
  içerir.

---

3. Detaylı Çalışma Mekanizması (Kritik Kısım):

A. Dil Türleri

- ***Makine dili:*** *0 ve 1'lerden oluşur*
- ***Assembly dili:*** *Makine dilinin sembolik hali*
- ***Yüksek seviyeli diller:*** *İnsan tarafından okunabilir*

B. Program Akışı ve Kontrol Mekanizması

Programlama dilleri şu yapıları içerir:

- ***Koşul yapıları:*** *(if, switch)*
- ***Döngüler:*** *(for, while)*
- ***Veri aktarımı:*** *Bellek ve değişkenler arası*

Bu yapılar, programın hangi adımda nasıl ilerleyeceğini belirler.

C. Programlama Paradigmaları

1. ***Yapısal Programlama:***
   - *Alt programlar (fonksiyon/prosedür) kullanılır*
2. ***Nesne Tabanlı Programlama (OOP):***
   - *Sınıf (class) ve nesne (object) kavramları*

D. Sınıf ve Nesne Kavramı

- ***Sınıf:*** *Soyut tanım (ör. insan, otomobil)*
- ***Nesne:*** *Sınıfın somut örneği*

Örnek:

- *"İnsan" → sınıf*
- *"Ali" → nesne*

E. Alt Programlar (Fonksiyon / Prosedür)

- *Belirli bir işi yapan bağımsız kod bloklarıdır*

Fonksiyon:

- *Değer döndürür*

Prosedür:

- *Genel işlemler yapar*

Rekürsiyon:

- *Alt programın kendini çağırması*

---

4. Teknik Detaylar ve İç Yapı:

- *Syntax (sözdizimi)*
- *Semantics (anlam)*
- *Derleyici (compiler)*
- *Yorumlayıcı (interpreter)*
- *Parametre aktarımı*
- *Stack kullanımı (fonksiyon çağrıları)*

---

5. Örnek (Adım Adım):

```
int kare(int x){
   return x * x;
}
```

1. *Fonksiyon çağrılır*
2. *Parametre stack'e alınır*
3. *ALU çarpma işlemi yapar*
4. *Sonuç geri döndürülür*

---

6. Avantajlar / Sınırlamalar:

Avantajlar:

- *Donanım bağımlılığını azaltır*
- *Hızlı geliştirme sağlar*

Sınırlamalar:

- *Performans kaybı (düşük seviyeye göre)*
- *Dil bağımlılığı*

VERİTABANLARI

1. Tanım:

Veritabanı, verilerin belirli bir düzen ve disiplin içerisinde saklanmasını, yönetilmesini ve hızlı bir şekilde sorgulanmasını sağlayan veri yönetim sistemidir.

---

2. Temel Mantık ve Çalışma Prensibi:

Temel amaç:

- *Veriyi organize etmek*
- *Hızlı erişim sağlamak*
- *Veri tekrarını azaltmak*

*Veriler genellikle **tablolar** halinde tutulur ve aralarında ilişkiler kurulur.*

---

3. Detaylı Çalışma Mekanizması:

A. Tablo Yapısı

- *Satır (record) → veri*
- *Sütun (field) → özellik*

B. Anahtar Kavramı

- *Veriye erişim için kullanılır*
- *Örn: ID*

C. Veritabanı Yönetim Sistemi (VTYS)

- *Veritabanını yönetir*
- *Uygulama ile veritabanı arasında arayüz sağlar*

D. İlişkisel Model

- *Tablolar arası ilişkiler kurulur*
- *En yaygın modeldir*

E. SQL (Structured Query Language)

- *Veri ekleme: INSERT*
- *Veri çekme: SELECT*
- *Güncelleme: UPDATE*
- *Silme: DELETE*

F. Veri Bağımsızlığı

- *Veritabanı yapısı değişse bile uygulama etkilenmez*

G. Nesne Tabanlı Veritabanı

- *Veriler nesne olarak tutulur*
- *İlişkiler nesne yapısı ile ifade edilir*

---

4. Teknik Detaylar:

- *Index yapıları*
- *Transaction yönetimi*
- *ACID prensipleri*
- *Sorgu optimizasyonu*

---

5. Örnek (Adım Adım):

*SELECT * FROM ogrenci WHERE id = 5;*

1. *Sorgu parser tarafından analiz edilir*
2. *Index varsa kullanılır*
3. *Veri diskten okunur*
4. *Sonuç kullanıcıya döndürülür*

---

6. Avantajlar / Sınırlamalar:

Avantajlar:

- *Hızlı veri erişimi*
- *Veri bütünlüğü*

Sınırlamalar:

- *Karmaşık yapı*
- *Yönetim maliyeti*

VERİ GÜVENLİĞİ

1. Tanım:

Veri güvenliği, bilgisayar sistemlerinde saklanan, iletilen veya işlenen verilerin yetkisiz erişim, değiştirme ve yok edilmelere karşı korunmasını sağlayan yöntemler bütünüdür.

---

2. Temel Mantık ve Çalışma Prensibi:

Temel amaçlar:

- *Gizlilik (Confidentiality)*
- *Bütünlük (Integrity)*
- *Erişilebilirlik (Availability)*

---

3. Detaylı Çalışma Mekanizması:

A. Şifreleme (Encryption)

- *Veri okunamaz hale getirilir*
- *Anahtar ile çözülür*

B. Saldırı Türleri

- *Virüs*
- *Worm (kurtçuk)*
- *Mantıksal bomba*

C. Güvenlik Önlemleri

- *Güçlü şifreler*
- *Erişim kontrolü*
- *Kullanıcı eğitimi*

D. Sistem İçi Riskler

- *İç tehditler (çalışanlar)*
- *Dikkatsizlik*

---

4. Teknik Detaylar:

- *Kriptografi*
- *Hash fonksiyonları*
- *Kimlik doğrulama*
- *Yetkilendirme*

---

5. Örnek (Adım Adım):

Bir parola saklama sistemi:

1. *Parola hashlenir*
2. *Veritabanında saklanır*
3. *Girişte tekrar hashlenir*
4. *Karşılaştırılır*

---

6. Avantajlar / Sınırlamalar:

Avantajlar:

- *Veri koruma*
- *Sistem güvenliği*

Sınırlamalar:

- *Performans maliyeti*
- *Yanlış yapılandırma riskleri*

ÖZET

*Bilgisayar mühendisliğinin temel amacı **algoritmik düşünme yeteneğini kazandırmaktır**.*
Bu bağlamda, donanım ve yazılım bilgisi birlikte ele alınmalıdır.

- *Donanım: CPU, bellek, I/O, ağlar*
- *Yazılım: algoritma, veri yapıları, işletim sistemleri*

Bir bilgisayar mühendisi:

- *Sistem tasarlayabilmeli*
- *Yazılım geliştirebilmeli*
- *Veriyi yönetebilmeli*
- *Güvenliği sağlayabilmelidir*

Bu disiplinler bir araya geldiğinde, modern bilgisayar sistemleri ortaya çıkar.